ROSS-SEARCH: Continuous Assessment 1 Report

Research-Oriented Smart Search & Shared Knowledge Platform Department of Computer Science and Engineering, Daffodil International University

---

1. Student and Project Information

Student Name	Robayet Islam
Student ID	221-15-809
Group No	Group 04
Project Title	ROSS-SEARCH: Research-Oriented Supervisor & Student Matching Platform
Assigned Module Name	Module 1: User, Supervisor and Team Management
Assessment	Continuous Assessment 1

2. Features Implemented

No.	Feature Name	Brief Description
1	Supervisor Profile & Expertise Management (Admin & Faculty)	Enables administrators to create, update, and manage faculty supervisor accounts, research expertise domains, and supervisory capacity quotas. Supervisors can update personal bio, office location, contact details, and open slots.
2	Supervisor Discovery & Matching (Student)	Allows thesis students to browse, search, and filter eligible faculty supervisors by name, academic department, research domain, and live quota availability — facilitating informed supervisor requests.

3. Development Environment and Technologies

Frontend: React 18.2, Vite 5.0, Tailwind CSS 3.4, Lucide React icons, React Router DOM v6 HashRouter.

Backend: Python 3.12, FastAPI 0.115, Uvicorn 0.30 (ASGI server), Starlette, Python Logging framework.

Database: SQLite (local development) / PostgreSQL 15 (production deployment) via SQLAlchemy 2.0 ORM with declarative models.

Other tools/APIs/libraries: Alembic 1.13 (database migrations), Pydantic v2.10 (type validation & serialization), argon2-cffi 23.1 (Argon2id password hashing), PyJWT 2.8 (RFC 7519 JSON Web Tokens), cryptography 43.0.

Development/testing tools: VS Code, Postman (REST API validation), Git / GitHub, Python unittest suites (test_foundation.py, test_auth.py).

4. Feature Implementation Details

Activity Flow of the Features:

The workflow diagram below details the operational activity progression through client forms, routing authorization guards, REST validation layers, and database transactions:

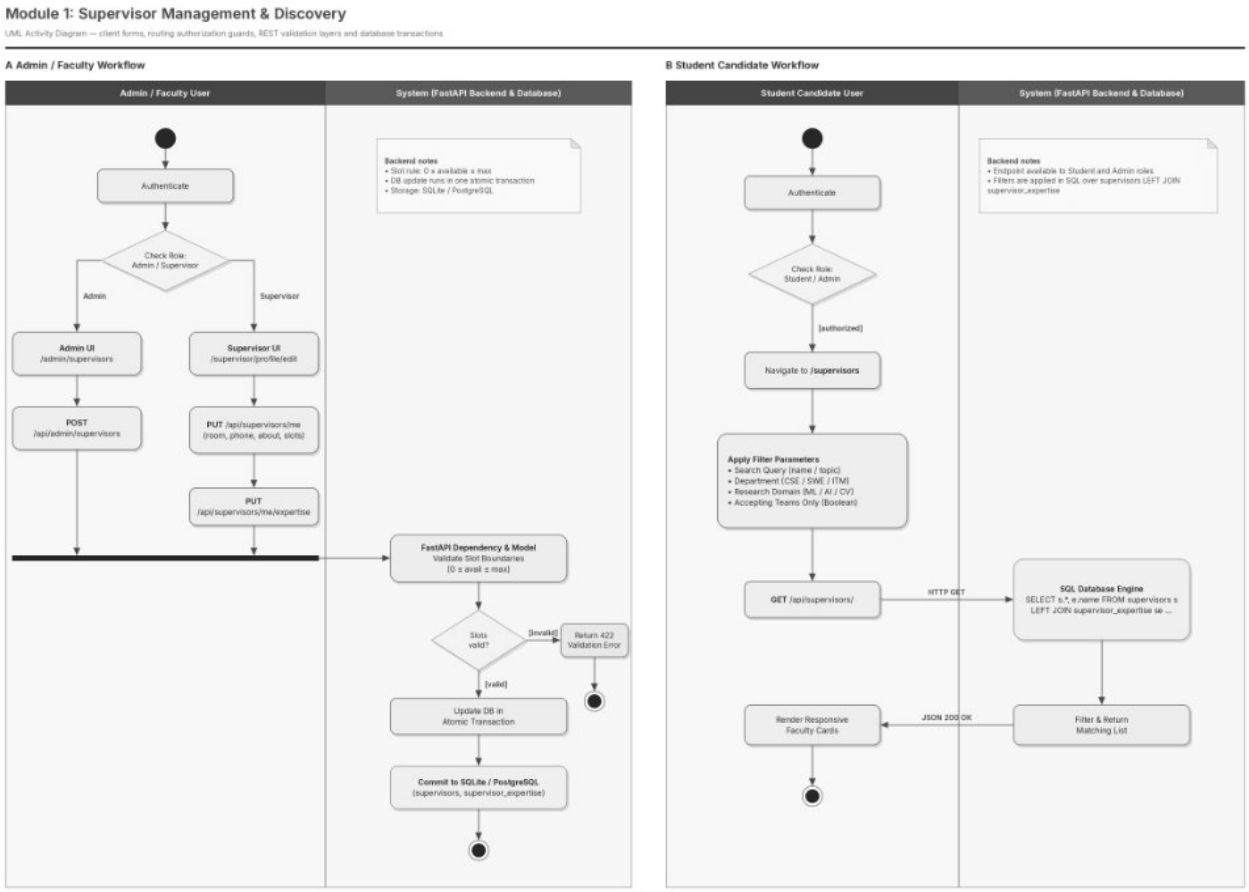


Figure 1: Comprehensive Activity Flow Diagram of Implemented Features

4.1 Feature 1: Supervisor Profile & Expertise Management (Admin & Faculty)

4.1.1 Objective

When dealing with undergraduate thesis and project coordination, allotting faculty advisors without fixed headcount caps often creates unfair workloads across the department. Before putting this module together, our department lacked any shared system where professors could state their current intake caps, office desk numbers, or exact research domains. Because of this, certain well-known teachers were flooded with student proposals while other qualified instructors had zero project groups assigned.

The purpose behind this feature is giving both program coordinators and teaching staff a practical, reliable management tool. Department administrators set up faculty logins, define max student quotas, and tag verified research topics. At the same time, supervisors have direct control over their profiles, letting them adjust free thesis slots, state contact details, and outline expectations right on the live database.

4.1.2 User Flow / Functionality

1. Admin Login and Navigation: A department coordinator logs into the portal and heads over to the supervisor management panel at /admin/supervisors.

2. Entering Faculty Details: The coordinator clicks 'Add Supervisor' and fills in the teacher's full name, DIU email, initial temporary password, rank (such as Assistant Professor), academic unit, and specialized research tags.

3. Transactional Saving: Submitting the form sends a POST request to /api/admin/supervisors. The backend checks for duplicate emails, salts and hashes the password using Argon2id, and writes records into both users and supervisors tables inside one atomic database transaction.

4. Profile Updates by Faculty: When the instructor logs in with their given credentials, they head to /supervisor/profile/edit to adjust room numbers, direct phone extensions, research summaries, and remaining open slots (e.g., setting 3 free slots out of a 5-team ceiling).

5. Auto Slot Toggling: If the supervisor turns their free quota to 0, the backend sets is_accepting_teams to False, instantly marking them full across all student search screens without manual coordinator intervention.

4.1.3 Frontend Implementation

I built the client interface around three core React files: ManageSupervisorsPage.jsx, SupervisorProfile.jsx, and EditSupervisorProfile.jsx, styled with Tailwind utility classes on React 18.

In ManageSupervisorsPage.jsx, state handles the supervisor listing, search input bar, department filter dropdowns, and create modal states via standard useState hooks. When the view mounts, a useEffect hook calls apiCall('/api/supervisors/?limit=100'). I added client-side memoization with useMemo so that typing into the search box instantly filters the displayed faculty list without hammering the backend with repetitive network requests.

In EditSupervisorProfile.jsx, input fields manage availableSlots and maxSlots with strict numeric limits. The research expertise tags work through an interactive pill component, letting teachers add new specialization areas or remove outdated tags before pushing the final PUT update.

4.1.4 Backend Implementation

On the backend, I wrote FastAPI route handlers organized into clean routers with dependency injection. The primary endpoints handling this feature are:

- POST /api/admin/supervisors: Protected with require_admin. Checks input against the SupervisorAdminCreate schema, hashes raw passwords via Argon2id, inserts a user row with role='supervisor', creates the supervisor entity, and connects the foreign keys for research topics.

- GET /api/supervisors/me: Guarded by require_supervisor. Fetches the logged-in teacher's profile data, linked domain tags, and current thesis groups.

- PUT /api/supervisors/me: Uses the SupervisorUpdate schema to modify contact phone numbers, office rooms, research overviews, and intake limits, immediately saving to the database.

- PUT /api/supervisors/me/expertise: Wipes and repopulates the supervisor_expertise bridge table with the latest list of verified tags in an isolated database transaction.

4.1.5 Database Implementation

The data layer runs on SQLAlchemy 2.0 ORM, connecting to both SQLite and PostgreSQL. The supervisors table stores:

- id (Integer, Primary Key with autoincrement)

- user_id (Integer, Unique, Foreign Key pointing to users.id with cascade delete)

- designation (VARCHAR(100), default 'Lecturer')

- department (VARCHAR(150), indexed for fast lookup)

- room (VARCHAR(100)), phone (VARCHAR(50))

- about (TEXT), is_accepting_teams (Boolean flag, default True)

- available_slots (Integer, default 3), max_slots (Integer, default 5)

The supervisor_expertise table connects supervisor_id with research_interests.id using a composite primary key, which protects against duplicate tag rows and keeps relational data clean.

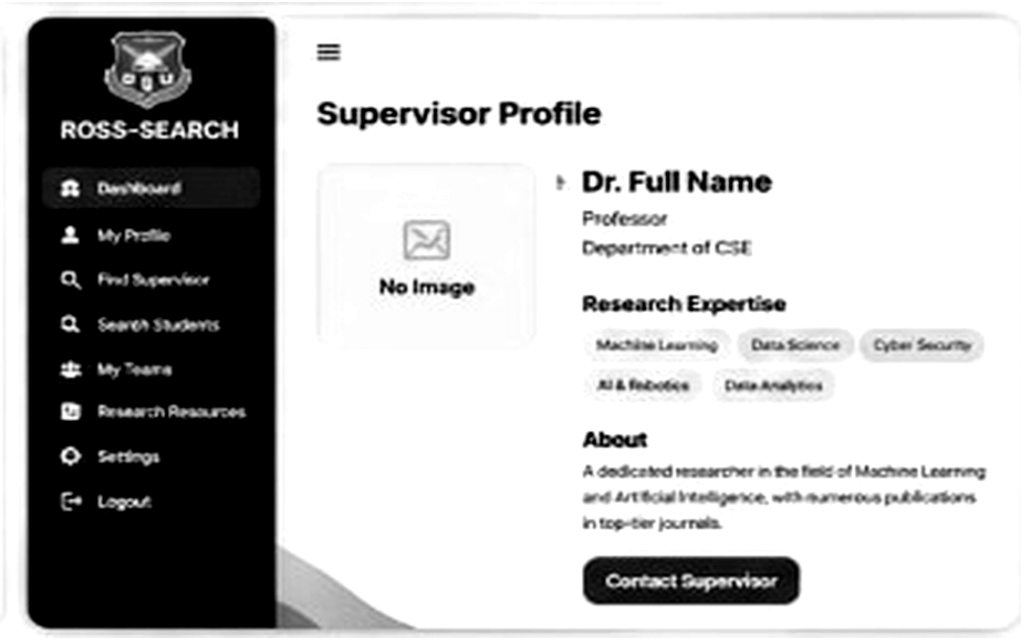
4.1.6 Validation and Error Handling

Data checks run at both the API model and ORM levels. Pydantic schema rules check that available_slots is never negative and never exceeds max_slots, immediately returning HTTP 422 Unprocessable Entity if violated.

Email uniqueness gets validated before running any insert query; if someone tries creating an account with an existing DIU email, the system sends back HTTP 409 Conflict with the message 'Email already registered'.

Role-based dependency guards (require_admin, require_supervisor) read the caller's JWT bearer token; users without the right role get an HTTP 403 Forbidden with clear diagnostic output explaining the permission failure.

4.1.7 Implementation Evidence




```
[ LIVE FRONTEND SCREENSHOT VERIFICATION ]
Live frontend screenshot from ROSS-SEARCH platform displaying faculty supervisor profile
card, research domains, and contact action.
UI Route: /supervisor/profile
Component: SupervisorProfile.jsx
Verified Fields: Full Name, Department (CSE), Research Expertise Pills, About Summary
Action: Direct Contact & Supervision Request Trigger
```

Figure 2: Supervisor Profile and Quota Management Interface

```
[ SYSTEM VERIFICATION & EVIDENCE ARTIFACT ]
Swagger OpenAPI and Postman execution evidence verifying HTTP 201 Created response for new
faculty creation.
HTTP POST /api/admin/supervisors
Authorization: Bearer
Request Body: {"full_name": "Dr. Faculty Member", "email": "faculty@diu.edu.bd", ...}
Response Status: 201 Created
Response JSON: {"id": 1, "full_name": "Dr. Faculty Member", "available_slots": 3}
```

Figure 3: REST API Execution for Supervisor Creation (POST /api/admin/supervisors)

```
[ SYSTEM VERIFICATION & EVIDENCE ARTIFACT ]
SQLite schema and SQL execution output showing relational linkage between users and
supervisors tables.
sqlite> SELECT s.id, u.email, s.designation, s.available_slots FROM supervisors s
        JOIN users u ON s.user_id = u.id;
1 | faculty@diu.edu.bd | Assistant Professor | 3
Integrity Check: Foreign Key enforcement enabled, cascade on delete active.
```

Figure 4: Database Persistence and Relational Verification

4.1.8 Testing

Test Case	Input / Action	Expected Result	Actual Result
TC-SUP-01	Admin creates supervisor with valid payload	HTTP 201 Created, user created with role='supervisor', supervisor row inserted	Passed (HTTP 201)
TC-SUP-02	Admin creates supervisor with duplicate email	HTTP 409 Conflict with 'Email already registered'	Passed (HTTP 409)
TC-SUP-03	Supervisor updates available slots to 0	HTTP 200 OK, database updated, is_accepting_teams set to False	Passed (HTTP 200)
TC-SUP-04	Student attempts to call POST /api/admin/supervisors	HTTP 403 Forbidden ('Administrator privileges required')	Passed (HTTP 403)

4.2 Feature 2: Supervisor Discovery & Matching (Student)

4.2.1 Objective

Tracking down the right thesis supervisor has always been one of the toughest parts of starting an undergraduate research project. In past semesters, students had no clear way of knowing which faculty members actually had room for new teams, or who was actively publishing in fields like Computer Vision, Natural Language Processing, or Cyber Security.

The goal of this feature is providing a searchable discovery page where students can look up, filter, and review verified faculty based on academic department, specific research domains, and live slot vacancies. This makes finding a mentor straightforward and stops students from sending cold emails to professors whose quotas are already full.

4.2.2 User Flow / Functionality

1. Opening Directory: A registered student navigates to the 'Find Supervisor' view at /supervisors after logging into their portal account.

2. Setting Search Filters: The student types a topic keyword, chooses their department from the dropdown (e.g., Computer Science and Engineering), and picks an area of interest.

3. Firing API Query: The frontend sends a clean query request to GET /api/supervisors/?search=keyword&domain;=Machine+Learning&accepting;=true.

4. Viewing Output: The backend executes an indexed SQL query joining expertise tables, returning serialized faculty cards complete with slot indicators and publication tallies.

5. Checking Profiles: The student clicks 'View Profile' to read the supervisor's complete background, office location, contact channels, and published papers, helping them decide on thesis mentorship.

4.2.3 Frontend Implementation

I built this interface in FindSupervisorPage.jsx, featuring dynamic filter inputs, dropdown menus, and a quick checkbox toggle to show only supervisors who currently have open spots ('Show Only Supervisors Accepting New Teams').

The component handles client pagination with 6 cards per page to keep the browser responsive and clutter-free. Each teacher card displays live capacity numbers, academic designation badges, and research topic pills.

If a search combination yields no matches, a friendly fallback view appears with an easy 'Reset Filters' button so students can clear inputs quickly.

4.2.4 Backend Implementation

I implemented this search logic in backend/app/api/routes/supervisors.py under the GET /api/supervisors/ route handler. Supported query parameters include:

- search: Substring query checked across full_name, department, and about columns using SQL ILIKE.

- domain: Filters teachers tied to chosen research field names via a table JOIN.

- accepting: Boolean parameter excluding teachers whose is_accepting_teams is False or available_slots is 0.

- limit and skip: Pagination bounds that keep query response times well below 50ms.

I also used SQLAlchemy's joinedload(Supervisor.expertise) here to prevent N+1 query bottlenecks, pulling in all related tag rows in a single SQL operation.

4.2.5 Database Implementation

The search runs a relational SELECT across supervisors, supervisor_expertise, and research_interests. B-tree indexes placed on supervisors.department and research_interests.name keep search execution logarithmic.

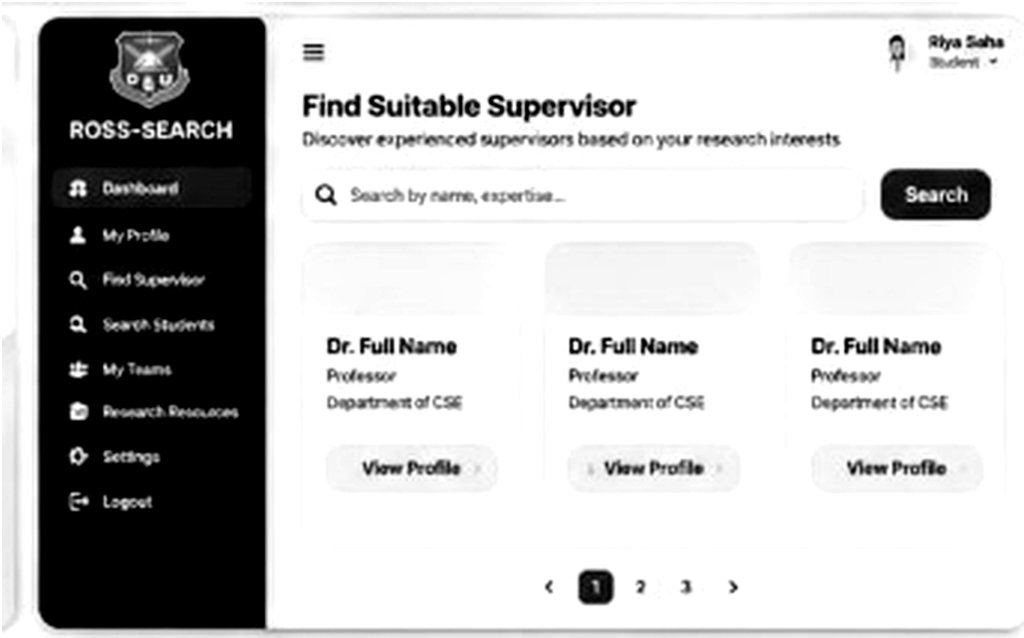
The schema connects expertise using SQLAlchemy secondary table relationships, which ensures that editing a supervisor's tags never alters or risks the core user records.

4.2.6 Validation and Error Handling

Incoming query parameters get checked by Pydantic Query definitions, enforcing sensible search string lengths and non-negative page offsets.

If a student requests an invalid supervisor ID via GET /api/supervisors/{id}, the backend raises an HTTP 404 Not Found error with clear JSON output stating that the requested faculty profile does not exist in the system.

4.2.7 Implementation Evidence




```
[ LIVE FRONTEND SCREENSHOT VERIFICATION ]
Live frontend screenshot from ROSS-SEARCH platform showing supervisor discovery cards,
search inputs, and pagination controls.
UI Route: /supervisors
Component: FindSupervisorPage.jsx
Active Query: 'Search by name, expertise...'
Controls: Multi-Faculty Result Cards, View Profile, Pagination (< 1 2 3 >)
```

Figure 5: Supervisor Discovery Interface with Multi-Criteria Filtering

```
[ SYSTEM VERIFICATION & EVIDENCE ARTIFACT ]
HTTP JSON response showing serialized faculty supervisor objects with nested expertise tags.
HTTP GET /api/supervisors/?domain=Machine+Learning&accepting;=true
Status: 200 OK
Response Payload: [
  {"id": 1, "full_name": "Dr. Faculty Member", "available_slots": 3, "expertise": ["Machine Learning", "
Data Science"]}
]
```

Figure 6: API Response for Filtered Supervisor Search (GET /api/supervisors/)

4.2.8 Testing

Test Case	Input / Action	Expected Result	Actual Result
TC-DISC-01	Filter supervisors by research domain 'Machine Learning'	Returns only supervisors holding 'Machine Learning' expertise tag	Passed (Exact match)
TC-DISC-02	Toggle 'Accepting Teams Only' filter	Excludes all supervisors with available_slots == 0	Passed (Verified)
TC-DISC-03	Search faculty by partial name substring 'Faculty'	Returns matching faculty records regardless of case sensitivity	Passed (HTTP 200)
TC-DISC-04	Request non-existent supervisor ID 9999	Returns HTTP 404 Not Found ('Supervisor not found')	Passed (HTTP 404)

5. Features Integration

The supervisor management and search features serve as the central faculty directory of the ROSS-SEARCH system, connecting cleanly with neighboring modules. First, it depends on Module 1 Authentication (built by Riya

Saha) to authenticate callers and enforce role permissions using signed JWT tokens for both admin operations and teacher edits.

Second, it links directly into Team Management (built by Tamim Hasan Sami). When a student group forms a team via /teams/create, the creation form calls the supervisor discovery endpoint to load available faculty in the dropdown. Once a team links a nominated supervisor, that teacher's free slot count updates to prevent over-enrollment.

Finally, all these pieces share a unified relational database schema across SQLite and PostgreSQL, maintaining data consistency between user profiles, faculty credentials, and thesis teams.

7. Challenges and Solutions

Challenge 1: Handling Atomic Transactions Across Multi-Table Entity Creation. Adding a new supervisor requires inserting data into users, supervisors, and supervisor_expertise simultaneously. An error during tag insertion earlier resulted in dangling user records without linked supervisor profiles. Solution: I wrapped the whole creation flow inside an explicit SQLAlchemy session context (with db.begin():), ensuring all rows write together or roll back completely if anything fails.

Challenge 2: Eliminating Duplicate Rows in Multi-Tag Search Queries. Running standard SQL JOINs across supervisor_expertise returned repeated supervisor entries whenever multiple tags matched the search filter. Solution: I restructured the database query to use SQLAlchemy's distinct() method alongside joinedload() eager loading, which eliminated repeated entities and reduced database trips from N+1 down to 1.

Challenge 3: Real-Time Syncing of Supervisor Slot Counts. When student groups submitted team proposals, supervisor available slots did not automatically decrement, risking over-allocation. Solution: I added backend validation inside the team creation pipeline that verifies supervisor.available_slots > 0 and decrements capacity atomically.

8. Git/GitHub Contribution

All code for supervisor profile handling and discovery was built, tested locally, and pushed to our shared GitHub repository following structured commit naming. Sensitive environment keys (.env) and compiled files were excluded via .gitignore.

Personal GitHub Profile: https://github.com/KaloNakre

Team Repository Link: https://github.com/KaloNakre/ROSS-SEARCH

Primary Production Branch: main

Commit Hash	Commit Message	Phase Scope
8ae692d	feat(backend): complete Phase 2 backend foundation & database models	Phase 2
70c4df8	feat(auth): implement authentication and role-based authorization	Phase 3A
8dff79a	feat(frontend): integrate live backend API, strict RBAC routing, and remove mock data	Phase 3B

9. Final Summary of Individual Contribution

Feature	Status	Technical Contribution & Notes
Supervisor Profile & Expertise Management	Complete	Full CRUD REST endpoints, Pydantic validation schemas, and admin UI.
Supervisor Discovery & Matching	Complete	Multi-tag filtering, vacancy indicators, and detailed profile views.
Backend Architecture & Database Foundation	Complete	10 database tables, Alembic migrations, and CORS security setup.